Reinforcement Learning

Policy Gradients

Policy Gradients?

Why Policy Gradients?

- Classical value-based RL has limitations:
  - **Difficult optimisation** in action space
  - **Instability** in continuous / high-dimensional domains
  - **Hard** to represent stochastic policies

Policy Gradients

- Policy gradient idea: optimize the policy directly
  - “A method that **directly optimizes** the policy by **adjusting** its **parameters** in the **direction** that **increases expected rewards**, using the **gradient** of the **objective** with **respect to the policy**.”

- Policy is parameterized as $\pi_\theta(a \mid s)$

Value Based vs Policy Gradients

- Value Based
  - Learns $Q(s, a)$ or $V(s)$
  - Action $= \arg\max Q$
  - Stuggles with continuos setting
  - Deterministic output
- Policy Gradients
  - Learns $\pi_\theta(a \mid s)$ directly
  - Action $=$ argmax$(Q \mid A), A \sim \pi$
  - Native continuous support
  - Stochastic or deterministic

Policy Gradients

- Why can't we just use value functions for everything?
- What does it mean to "optimise a policy directly"?
- How does a neural network become a policy?

Policy Gradients

- **Advantages** of **Policy Gradients** over value-based methods:

  - Naturally handles continuous actions
  - Supports stochastic or deterministic policies
  - Can represent multi-modal behaviors
  - Flexible architectures: CNNs, RNNs, Transformers, GNNs

Objective: maximize expected return $J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)]$

How ?

State input ⟶ [MLP / CNN / RNN / Transformer] ⟶ Action output

Policy Network $\pi_\theta(a|s)$

(discrete: softmax → $P(a|s)$) | (continuous: μ, $\sigma \rightarrow N(\mu,\sigma^2)$)

- Discrete actions: softmax over logits.
- Continuous: predict mean μ and log-std σ, then sample.

Policy Network Architectures

Architecture Choices

MLP — Low-dimensional state vectors (CartPole, MuJoCo)

CNN — Pixel / image observations (Atari, robot cameras)

RNN / LSTM — Partial observability — POMDP settings

Transformer — Sequential & multi-agent tasks

GNN — Graph-structured environments

Output Heads

Stochastic — Continuous (PPO, SAC)

$f\theta(s) \rightarrow [\mu, \log\sigma] \rightarrow a \sim N(\mu, \sigma^2)$

Sample from Gaussian; supports multi-modal behaviour

Deterministic (DDPG, TD3)

$f\theta(s) \rightarrow a \quad \text{(no sampling)}$

Lower variance; needs explicit exploration noise

Discrete (A2C, DQN)

$f\theta(s) \rightarrow \text{softmax} \rightarrow P(a|s)$

Categorical distribution over finite action set

Actor and Critic Methods

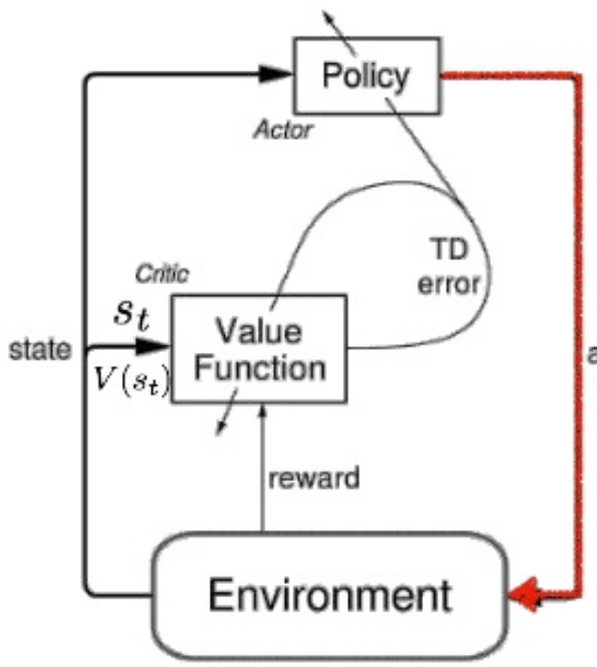
- **Actor**: policy network that selects actions
  - $\pi_\theta(a \mid s)$
  - Input: state s , Output action a
- **Critic:** value network that evaluates states or state–action pairs
  - $V_\phi(s)$
  - $Q_\phi(s, a)$
  - Input state s, Output value V
  - Provides signal δ_t (TD-error) to update policy gradient
- Notation: π_θ = actor policy, V_ϕ = critic value function, δ_t= TD-error signal)

Actor and Critic Methods

- **Critic** computes **Temporal difference (TD)-error** as learning signal:
  - $\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$
- **Actor** Updates direction proportional to:
  - $\theta \leftarrow \theta + \alpha\, \delta_t\, \nabla_\theta \log \pi_\theta(a_t \mid s_t)$
    - $\delta_t > 0$: action was better than expected — reinforce it.
    - $\delta_t < 0$: discourage it.

Actor Critic

- Actor–Critic combines:
  - Actor: policy model $\pi_\theta(a \mid s)$
  - Critic: value model $V\phi(s)$ and $Q\phi(s, a)$
- Actor improves the policy
- Critic evaluates actions using value function
- Learning signal = **advantage** or **TD-error**
  - **Actor update guided by value estimate**
  - **Critic stabilises policy gradient**



- Actor: decides which action to take
- Critic: tells the actor how good its action was and how it should adjust

Critic Value Estimation

- Critic predicts state values and action value
  - $V^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid s_t = s]$
- Loss function ?
  - Critic state value
    - $L_{\phi} = \mathbb{E}_t \left[(V_{\phi}(s_t) - (r_t + \gamma V_{\phi}(s_{t+1})))^2 \right]$
    - Reduce **TD-error** via regression
  - Critic action value, for Q-based
    - $L_{\phi} = \mathbb{E}_t \left[(Q_{\phi}(s_t, a_t) - (r_t + \gamma \max_{a'} Q_{\phi}(s_{t+1}, a')))^2 \right]$

Actor Update with Critic Signal

- Actor uses critic's estimate to update policy
- Policy gradient with TD-error:
  - $\nabla_\theta J(\theta) = \mathbb{E}\left[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\, \delta_t\right]$
- Clear separation:
  - Critic reduces variance
  - Actor improves performance

Policy Gradient Objective

- **Goal:** directly optimize the expected return
- The agent samples trajectories τ using current policy
- Objective function:
  - $J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)]$
- Policy is differentiable w.r.t. parameters θ

Policy Gradients

- Policy gradient theorem provides a usable expression
- Gradient of expected return is:
  - $\nabla_\theta J(\theta) = \mathbb{E}\left[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\, Q^\pi(s_t, a_t)\right]$
- No need to differentiate the state distribution
- **Core formula used** in **all policy gradient** and **actor–critic** methods

Advantage Function

- Advantage measures how much better an action was vs. expected
- Defined as:
  - $A(s, a) = Q(s, a) - V(s)$
- Using advantage gives the optimal baseline
- Advantage-weighted policy gradient:
  - $\nabla_\theta J(\theta) = \mathbb{E}\left[\nabla_\theta \log \pi_\theta(a \mid s)\, A(s, a)\right]$
- Critic estimates $V(s)$ or both $Q(s, a)$ and $V(s)$

Generalised Advantage Estimation (GAE)

- Formulation: $\hat{A}_t^{\text{GAE}(\gamma,\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}$
  - Where $\delta_{t+l} = r_{t+l} + \gamma V(s_{t+l+1}) - V(s_{t+l})$
  - λ is the bias variance key!

λ value	Estimator	Bias	Variance
λ = 0	TD(0)	High	Low
λ = 1	Monte Carlo	None	Very High
λ ≈ 0.95	**GAE (PPO default)**	**Tuneable**	**Tuneable ★**

- **Single formula unifies all advantage estimators.** By tuning λ you control the bias-variance trade-off without changing your algorithm. Used in PPO, A2C, and most modern on-policy methods.

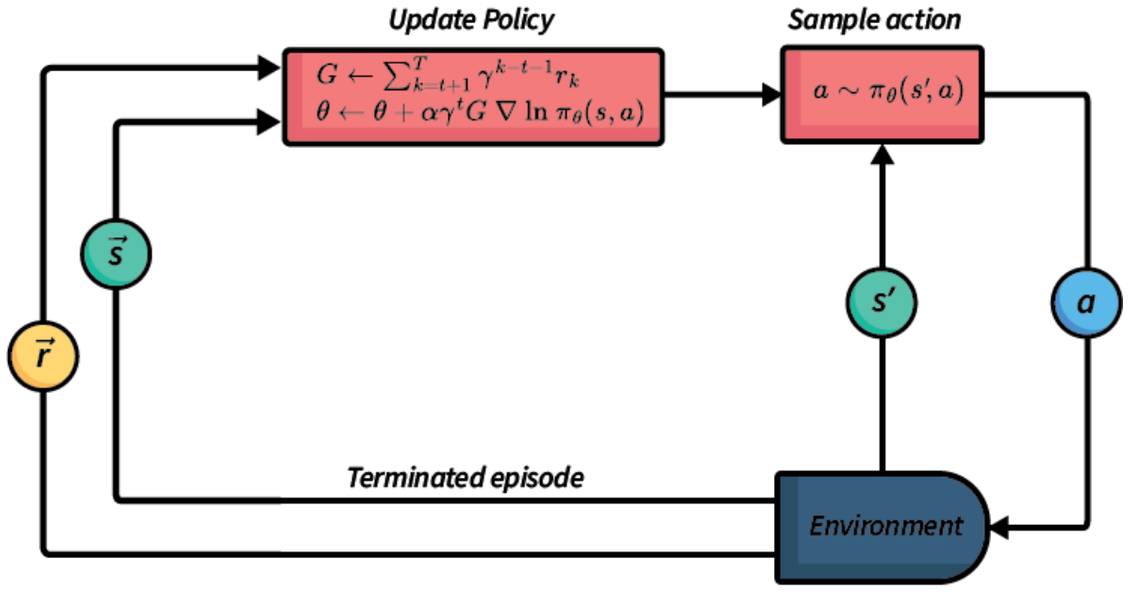
Why Actor-Critic Instead of Pure Policy Gradient?

- REINFORCE (pure policy gradient):
  - High variance, slow convergence
  - Sensitive to reward scale
  - Gradient estimate (REINFORCE):
    - $\nabla_\theta J(\theta) = \mathbb{E}\left[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\, G_t\right]$
- Introduce a baseline via the advantage:
  - $G_t = A(s,a) = Q(s,a) - V(s)$
- Advantage-based policy gradient:
  - $\nabla_\theta J(\theta) = \mathbb{E}\left[\nabla_\theta \log \pi_\theta(a \mid s)\, A(s,a)\right]$

Overall View on RL

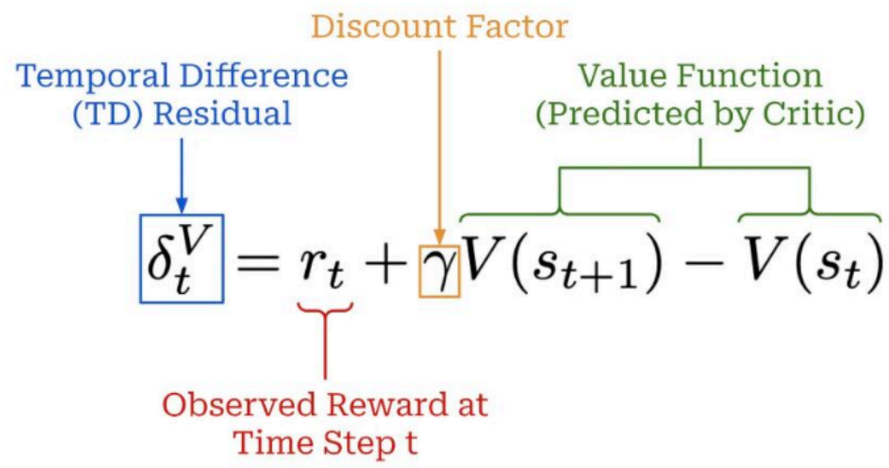
View on Policy Gradient

- All modern deep RL methods fundamentally optimize:
  - $\nabla_\theta J = \mathbb{E}\left[\nabla_\theta \log \pi_\theta(a \mid s)\, A(s,a)\right]$
  - Differences arise from:
    - How **advantage** $A(s,a)$ is estimated
    - How **policy updates** are constrained
    - How **critics** are trained
    - Whether **stochastic** or **deterministic** policies are used
    - Whether training is **on-policy** or **off-policy**



View on Policy Gradient

- All Actor–Critic methods use some variant:
  - $A(s_t, a_t) \approx \sum_{k=0}^{K} (\gamma\lambda)^k \delta_{t+k}$



- Special cases:
  - REINFORCE: $K = T - t, \lambda = 1$
  - TD(0): $K = 0, \lambda = 0$
  - GAE: arbitrary $\lambda \in [0,1]$

General Policy Update Rule

- General update:
  - $\theta \leftarrow \theta + \alpha\, g$
- Where:
  - $g = \mathbb{E}\left[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\, A_t\right]$
- Constraints or penalties define each algorithm!

Trust Region Policy Optimization (TRPO)

- **TRPO** is a policy gradient method that updates the policy while **guaranteeing monotonic improvement** by enforcing a trust region constraint using **KL divergence**.

- $\max_{\theta} \quad \mathbb{E}[r_t A_t]$

- According to $D_{\text{KL}}(\pi_{\theta_{\text{old}}} \parallel \pi_{\theta}) \leq \delta$

Proximal Policy Optimization

- PPO is a **policy gradient algorithm** that stabilizes updates by **clipping the policy ratio**, preventing destructive large steps (similar motivation as TRPO), but **without complicated constrained optimization**.

- $L_{\text{CLIP}} = \min\left(r_t A_t,\ \text{clip}(r_t, 1-\epsilon, 1+\epsilon) A_t\right)$

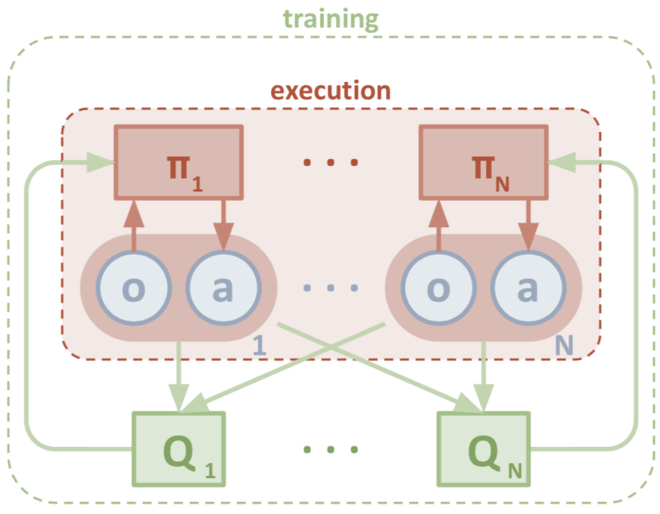
- It approximates TRPO while using a soft trust region defined by the clip

Deterministic Policy Gradient Family

- We did this in the practical class
  - $\nabla_\theta J = \mathbb{E}\left[\nabla_\theta \mu_\theta(s)\, \nabla_a Q(s,a)\Big|_{a=\mu_\theta(s)}\right]$
    - Off-policy learning with **replay buffers** and **target networks**.
- Variants:
  - DDPG - Deep Deterministic Policy Gradient
  - TD3 - Twin Delayed Deep Deterministic Policy Gradient
  - MADDPG — **Multi-Agent** DDPG

Deterministic Policy Gradient Family

Algorithm	Setting	Key Ideas	Fixes
DDPG	Single agent, continuous	Deterministic actor–critic, target networks	Baseline for continuous RL
TD3	Single agent, continuous	Twin critics, delayed updates, smoothing	Overestimation, instability
MADDPG	Multi-agent (N agents)	One actor per agent + centralized critic	Non-stationarity in multi-agent RL

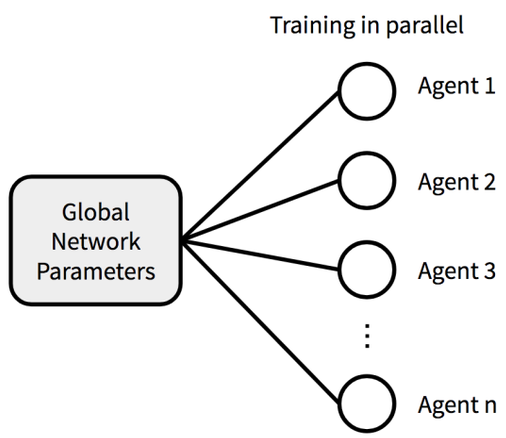


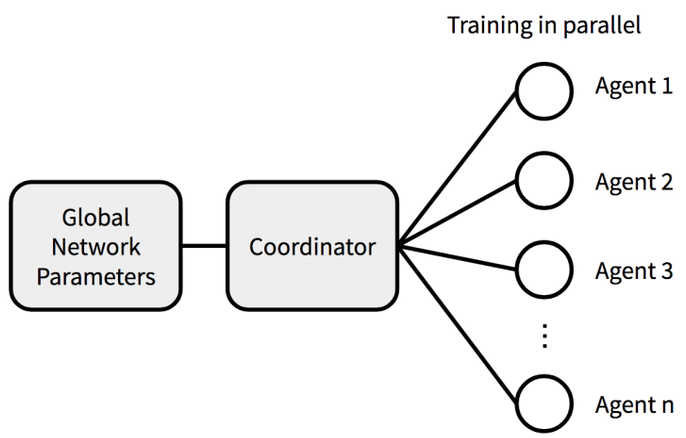
Soft Actor–Critic (SAC) as Soft Policy Gradient

- SAC maximizes:
  - $J = \mathbb{E}[Q(s,a)] + \alpha \mathscr{H}(\pi)$
- Soft Q-function:
  - $V(s) = \mathbb{E}_{\pi}[Q(s,a) - \alpha \log \pi(a \mid s)]$
- Produces robust stochastic policies.

Advantage Actor–Critic and Asynchronous Advantage Actor–Critic

- A2C/A3C = synchronized/ asynchronized actor–critic using:
  - $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$
- Multiple point of view and parallel works (like the Pokemon RL video!)





AC unified view

- All deep actor–critic methods minimize:
  - $L(w) = \frac{1}{2}\left(Q_w(s,a) - y\right)^2$
- Where target is:
  - TD target (DDPG/TD3/SAC)
  - n-step return (A2C/A3C)
  - Soft Q target (SAC)

On-Policy vs Off-Policy

- On-Policy : if it **must learn from data generated** by the **current policy** being optimised.
    - Stable
    - Less sample-efficient
- Off-Policy: if it can learn from **any behavior policy**, not just the current one.
    - Highly sample-efficient
    - Requires replay buffer
    - Harder to tune

Common Stability Mechanisms Across Methods

- All modern RL uses:
    - Target networks
    - Normalization
    - Entropy bonuses
    - Clipping (PPO)
    - Clipped double Q-learning (TD3/SAC)
    - GAE for advantage estimation
    - Replay buffers (off-policy)

Algorithm Selection Guide

PPO *On-policy*

When: Default on-policy choice

Best for: Stable, easy to tune, works for most tasks

SAC *Off-policy*

When: Continuous control, SOTA stability

Best for: Sample-efficient, robust stochastic policies

TD3 *Off-policy*

When: Deterministic continuous control

Best for: Less stochastic than SAC, strong baseline

A2C / A3C *On-policy*

When: Distributed on-policy training

Best for: Multiple parallel workers, fast wall-clock time

DDPG *Off-policy*

When: Simple deterministic policy baseline

Best for: Starting point for continuous control research

Decision Transformer *Offline*

When: Offline RL from fixed datasets

Best for: No environment interaction needed during training

Unifying Theory: Policy Improvement

- Seems logic and trivial but hard to grasp:
  - $J(\pi_{\text{new}}) \geq J(\pi_{\text{old}})$
- Either via:
  - Trust region
  - Clipping
  - Advantage weighting
  - Conservative Q-updates

What really matters?

- Good function approximation
- Stable target values
- Proper advantage estimation
- Regularized policy updates
- Strong exploration (entropy, noise)
- Large-scale data collection
- Good engineering (normalization, clipping)

The **Final RL** Equation

- At the core, RL is always:
  - Policy Update + Value Update
  - Policy Update
    - $\nabla_\theta J \propto \nabla_\theta \log \pi_\theta A$
  - Value Update
    - $Q \leftarrow r + \gamma V(s')$

In the end…

- All Actor–Critic algorithms are variations of a *single unifying framework*
- Differences lie in:
    - Update constraints
    - Advantage estimation
    - Critic structure
    - Exploration strategy
    - On-policy vs off-policy
- Despite diversity → unified mathematical structure

Summary

- Policy gradients are the backbone of modern RL
- Advantage estimation = stability
- Critic quality = performance
- Deep networks = expressivity
- Trust regions, entropy, double critics = robustness
- Distributed RL = scalability
- Offline RL + Transformers = future direction

Key Contributions

1992	**REINFORCE**	Simple statistical gradient-following algorithms for connectionist RL	*Williams*
2000	**PG Theorem**	Policy gradient methods for RL with function approximation	*Sutton et al.*
2015	**GAE**	High-dimensional continuous control using generalised advantage estimation	*Schulman et al.*
2015	**TRPO**	Trust region policy optimization	*Schulman et al.*
2016	**A3C**	Asynchronous methods for deep reinforcement learning	*Mnih et al.*
2017	**PPO**	Proximal policy optimization algorithms	*Schulman et al.*
2018	**SAC**	Soft actor-critic: off-policy max entropy deep RL	*Haarnoja et al.*
2018	**TD3**	Addressing function approximation error in actor-critic methods	*Fujimoto et al.*

■ **Foundation** ■ **Actor-Critic** ■ **On-Policy**

Recent Trends

RL Discovering Algorithms

- DeepMind systems using RL to discover new algorithms
- AlphaTensor: new matrix multiplication algorithms
- AlphaDev: faster sorting algorithms
- RL framed algorithm discovery as a game/search problem

RL for Reasoning in LLMs

- Reinforcement learning used to improve reasoning ability
- Methods: RLHF, RLVR (Reinforcement Learning with Verifiable Rewards)
- Used in modern reasoning models
- Rewards computed from correctness (math, code, logic)
- Enables self-improvement after pretraining

World-Model RL

- Agents learn an internal model of the environment
- Planning performed inside a latent representation
- Important systems:
  - DreamerV3
  - EfficientZero
- Improves sample efficiency and generalization

Meta-RL Discovering Learning Rules

- AI systems learning better RL algorithms automatically
- Meta-learning over many environments
- Instead of designing update rules manually
- RL learns the RL optimizer

RL for Robotics with Foundation Models

- Vision-Language-Action (VLA) models
- Robots learn from demonstrations + exploration
- Combines perception, language, and control
- Reduces need for massive labeled datasets

Big Picture

- RL moving beyond games
- Integration with foundation models
- Discovery of scientific knowledge
- More sample-efficient and general agents
- Increasing role in reasoning systems

Main Takeways

- RL is expanding into discovery, reasoning, and robotics
- Combination with large models is a key direction
- World models and algorithm discovery are major research frontiers
- Potential impact across science, engineering, and AI systems

Demos, Code and Resources

Code Libraries

CleanRL

Single-file PPO/SAC/TD3 — best for teaching & understanding

Stable Baselines 3

Production-ready implementations, clean API

Spinning Up (OpenAI)

Well-documented PPO, SAC, VPG with theory

Sample Factory

Ultra-fast distributed A2C/PPO

Courses & Tutorials

HuggingFace Deep RL Course

Hands-on PPO, A2C, DQN — free, with notebooks

CS285 UC Berkeley (Levine)

Lectures 5–6: policy gradients & actor-critic in full depth

UCL RL Course (Silver)

Lectures 7–8: policy gradient & actor-critic foundations

Lilian Weng Blog

Policy gradient walkthrough — already referenced in slides

```
PPO in 3 lines:  model = PPO('MlpPolicy', env)  →  model.learn(500_000)  →  model.predict(obs)
```

References

- Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. Machine Learning, 8, 229–256.

- Schulman, J., Levine, S., Abbeel, P., Jordan, M., & Moritz, P. (2015). Trust region policy optimization. ICML.

- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. arXiv:1707.06347.

- Schulman, J., Moritz, P., Levine, S., Jordan, M., & Abbeel, P. (2015). High-dimensional continuous control using generalized advantage estimation. ICLR.

- Mnih, V., et al. (2016). Asynchronous methods for deep reinforcement learning. ICML.

- Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). Soft actor-critic. ICML.

- Fujimoto, S., Hoof, H., & Meger, D. (2018). Addressing function approximation error in actor-critic methods. ICML.

References

- Christopher M. Bishop and Hugh Bishop, Deep Learning - Foundations and Concepts (2024). Springer 2024,
- Prince, S. J. (2023). Understanding Deep Learning. MIT Press.
- Drori, I. (2022). The Science of Deep Learning. Cambridge University Press.
- Brunton, S. And Kutz, J. N. (2017). Data Driven Science & Engineering - Machine Learning, Dynamical Systems, and Control
- https://lilianweng.github.io/posts/2018-04-08-policy-gradient/#:~:text=%24A,value%20off%20as%20the%20baseline
- Richard S. Sutton and Andrew G. Barto. Reinforcement Learning: An Introduction; 2nd Edition. 2017.

References

- https://www.youtube.com/watch?v=1kV-rZZw50Q
- https://www.youtube.com/watch?v=Mz7Qp73lj9o
- https://www.youtube.com/watch?v=xiChGsduWEU
- https://www.youtube.com/watch?v=WMmGzx-jWvs